

Міністерство освіти і науки України
Національний технічний університет України “Київський політехнічний інститут
імені Ігоря Сікорського”
Кафедра обчислювальної техніки ФІОТ

ЗВІТ
з лабораторної роботи №1
з навчальної дисципліни «Технології Data Science для завдань електронної комерції»

Тема:

“ПІДГОТОВКА ТА АНАЛІЗ ДАНИХ ДЛЯ СТАТИСТИЧНОГО НАВЧАННЯ”

Виконав:

Студент 1 курсу кафедри ІІІ ФІОТ
Навчальної групи ІІІ-51мн
Сергій ПАНЧЕНКО

Перевірив:

Професор кафедри ОТ ФІОТ
Олексій ПИСАРЧУК

1 Мета

Виявити, дослідити та узагальнити особливості застосування методів статистичного навчання для задач визначення статистичних характеристик вхідного потоку даних з використанням спеціалізованих пакетів мови програмування Python.

2 Завдання

Лабораторія провідної IT-компанії реалізує масштабний проект розробки універсальної платформи з обробки Big Data масиву статистичних даних поточного спостереження для виявлення закономірностей і прогнозування розвитку контролюваного процесу. Платформа передбачає розташування back-end компоненти на власному хмарному сервері з наданням повноважень користувачам заздалегідь адаптованого front-end функціоналу універсальної платформи.

Замовниками ресурсів платформи є: державні та комерційні компанії валютного трейдингу для прогнозування динаміки зміни курсу валют та ціни інших товарів; метеорологічні служби для прогнозування параметрів метеоумов; департаменти охорони здоров'я для прогнозування зміни показників епідеміологічних ситуацій тощо.

Вам, як Data Science Engineer поставлено завдання.

Завдання III рівня – максимально 9 балів.

1. Провести парсинг самостійно обраного сайту. Вміст даних, що підлягають парсингу – обрати самостійно.
2. Результати парсингу зберегти у файлі. Тип файлу обрати самостійно.
3. Оцінити динаміку тренду реальних даних.
4. Здійснити визначення статистичних характеристик результатів парсингу.
5. Синтезувати та верифікувати модель даних, аналогічних за трендом і статистичними характеристиками реальним даним, які є результатом парсингу.
6. Провести аналіз отриманих результатів.

3 Результати виконання лабораторної роботи.

3.1 Синтезована математична модель перетворень графічних об'єктів відповідно до індивідуального завдання.

Синтезована математична модель для Завдання III рівня включає в себе модель динаміки популяції населення Землі (парсинг сайту worldometers.info).

Використовується адитивна модель:

$$Y(t) = \text{Trend}(t) + \text{Noise}(t)$$

1. **Модель ідеального тренду:** Для апроксимації реальних даних використано поліноміальний тренд 2-го ступеня (квадратичний закон), параметри якого визначаються методом найменших квадратів (МНК):

$$\text{Trend}(t) = a_0 + a_1 t + a_2 t^2$$

2. **Модель генерації випадкової величини (шуму):** Для імітації відхилень від ідеального тренду (залишків) використовується нормальний закон розподілу $N(0, \sigma^2)$. Стандартне відхилення σ обчислюється як середньоквадратичне відхилення залишків між реальними даними та лінією тренду.

$$f(x, m_x, \sigma_x) = \frac{1}{\sqrt{2\pi}\sigma_x} \exp\left(-\frac{(x - m_x)^2}{2\sigma_x^2}\right)$$

де математичне сподівання шуму $m_x = 0$.

3.2 Блок схема алгоритму та її опис.



Алгоритм розробленого програмного скрипта:

1. **Ініціалізація:** Імпорт необхідних бібліотек (requests, bs4, pandas, numpy, matplotlib).

2. **Парсинг даних:** HTTP-запит (GET) до сайту `worldometers.info`. Обробка HTML-структури за допомогою `BeautifulSoup`. Вилучення даних про рік та кількість населення.
3. **Збереження:** Запис розпарсених даних у файл `parsed_population.csv`.
4. **Статистичний аналіз:** Обчислення математичного сподівання, дисперсії та СКВ для реальних даних.
5. **Побудова тренду:** Використання функції `numpy.polyfit` для підбору коефіцієнтів полінома 2-го ступеня.
6. **Визначення характеристик шуму:** Обчислення залишків (реальні дані мінус тренд) та їх СКВ.
7. **Синтез даних:** Генерація нормального шуму із розрахованим СКВ та нульовим мат. сподіванням. Додавання шуму до ідеальної лінії тренду.
8. **Візуалізація:** Збереження графіків (динаміка тренду та гістограми) у папку `build`.

3.3 Опис структури проекту програми.

Структура проекту:

- `level3.py` — головний файл програмного коду, що виконує парсинг, розрахунки та синтез.
- `build/` — директорія, що створюється автоматично для збереження результатів.
 - `parsed_population.csv` — файл із результатами парсингу.
 - `trend_analysis.png` — графік аналізу тренду.
 - `comparison_histograms.png` — графіки гістограм.

3.4 Результати роботи програми відповідно до завдання.

Програма успішно розпарсила дані, обчислила лінію тренду та синтезувала нову вибірку.

Статистичні характеристики розпарсених (реальних) даних:

- Математичне сподівання: 4 392 219 101.84
- Дисперсія: 5 799 597 087 186 308 096.00
- СКВ: 2 408 235 264.08

Статистичні характеристики залишків тренду (шуму):

- Математичне сподівання: 0.00
- Дисперсія: 2 316 161 339 172 337 152.00
- СКВ: 1 521 893 997.35

Статистичні характеристики синтезованих даних:

- Математичне сподівання: 4 181 523 057.66
- Дисперсія: 5 814 514 104 467 358 720.00
- СКВ: 2 411 330 359.87

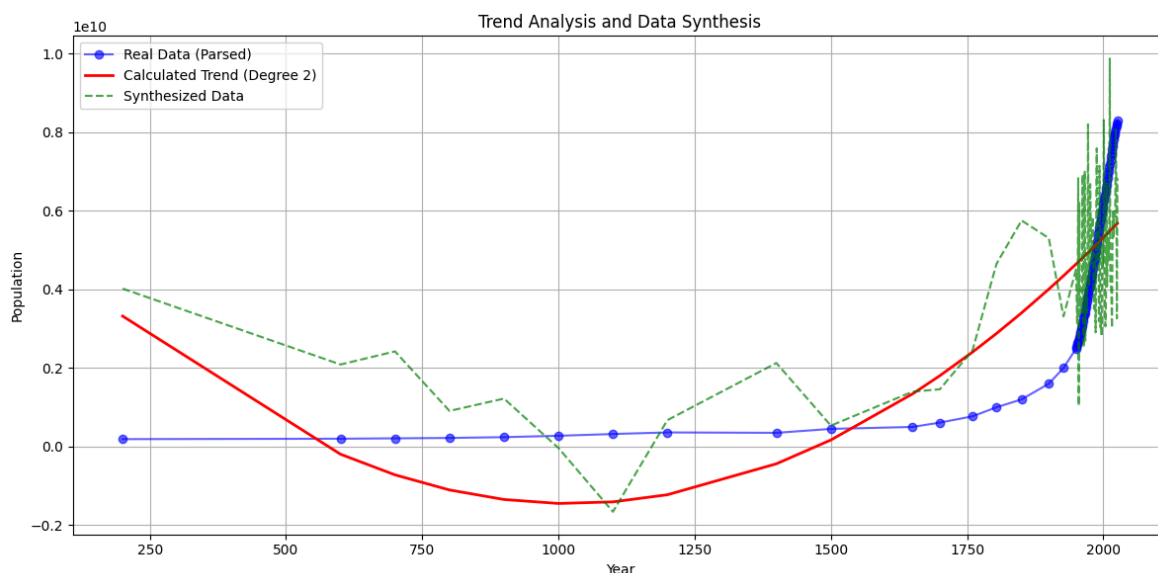


Рисунок 3.1 — Аналіз динаміки тренду та синтезована вибірка

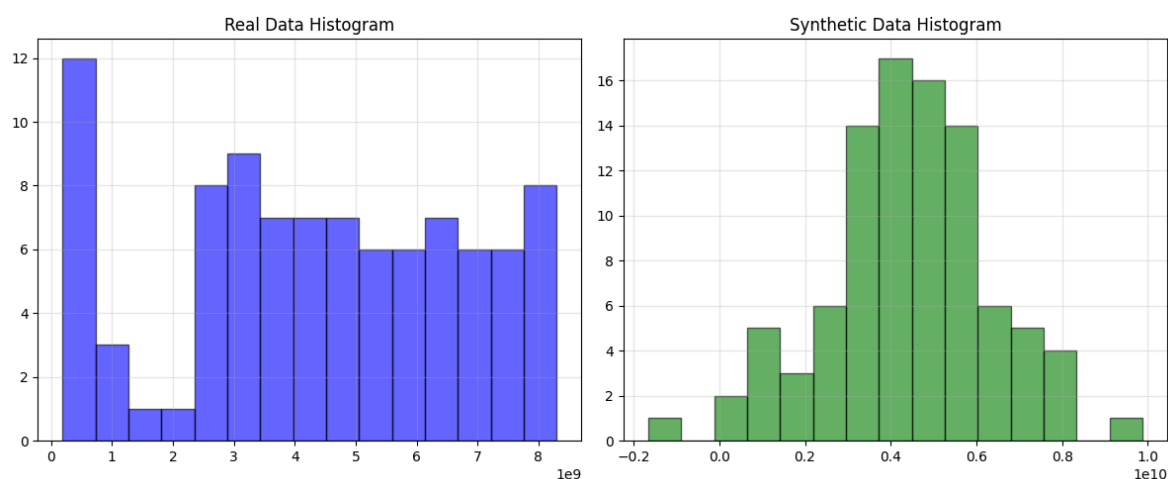


Рисунок 3.2 — Порівняння гістограм реальних та синтезованих даних

3.5 Програмний код.

Програмний код послідовно реалізує алгоритм та спрямовано на отримання результатів, поданих вище. При цьому використано можливості Python бібліотек: requests, bs4, pandas, numpy, matplotlib.

3.5.1 level3.py

```
import os
import pathlib
import requests
import bs4
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import dataclasses
```

```
SCRIPT_PATH: pathlib.Path = pathlib.Path(os.path.abspath(__file__))
SCRIPT_DIR: pathlib.Path = SCRIPT_PATH.parent
BUILD_DIR: pathlib.Path = SCRIPT_DIR / 'build'
```

```

BUILD_DIR.mkdir(parents=True, exist_ok=True)

@dataclasses.dataclass
class PopulationRecord:
    year: int
    population: int

def parse_world_population(url: str) -> list[PopulationRecord]:
    headers: dict[str, str] = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36'
    }
    response: requests.Response = requests.get(url, headers=headers)
    response.raise_for_status()

    soup: bs4.BeautifulSoup = bs4.BeautifulSoup(response.text, 'html.parser')
    table: bs4.element.Tag | None = soup.find('table')

    records: list[PopulationRecord] = []

    if table:
        tbody: bs4.element.Tag | None = table.find('tbody')
        if tbody:
            for row in tbody.find_all('tr'):
                cols: bs4.ResultSet = row.find_all('td')
                if len(cols) >= 2:
                    year_str: str = cols[0].text.strip()
                    pop_str: str = cols[1].text.strip().replace(',', '')
                    try:
                        records.append(PopulationRecord(
                            year=int(year_str),
                            population=int(pop_str)
                        ))
                    except ValueError:
                        continue

    records.sort(key=lambda x: x.year)
    return records

def save_to_csv(records: list[PopulationRecord], filepath: pathlib.Path) ->
None:
    df: pd.DataFrame = pd.DataFrame([r.__dict__ for r in records])
    df.to_csv(filepath, index=False)

def calculate_statistics(data: np.ndarray, label: str) -> tuple[float, float,
float]:
    mean_val: float = float(np.mean(data))
    var_val: float = float(np.var(data))
    std_val: float = float(np.std(data))

    print(f"--- {label} ---")
    print(f"Expected Value: {mean_val:.4f}")
    print(f"Variance: {var_val:.4f}")
    print(f"Std Dev: {std_val:.4f}")

```

```

print("-" * 30)

return mean_val, var_val, std_val

def fit_polynomial_trend(x: np.ndarray, y: np.ndarray, degree: int = 2) ->
np.ndarray:
    coeffs: np.ndarray = np.polyfit(x, y, degree)
    trend: np.ndarray = np.polyval(coeffs, x)
    return trend

def synthesize_data(trend: np.ndarray, std_dev: float) -> np.ndarray:
    noise: np.ndarray = np.random.normal(0.0, std_dev, len(trend))
    return trend + noise

def plot_trend_analysis(x: np.ndarray, y_real: np.ndarray, y_trend: np.ndarray,
y_synthetic: np.ndarray, filepath: pathlib.Path) -> None:
    plt.figure(figsize=(12, 6))
    plt.plot(x, y_real, label='Real Data (Parsed)', marker='o', color='blue',
alpha=0.6)
    plt.plot(x, y_trend, label='Calculated Trend (Degree 2)', color='red',
linewidth=2)
    plt.plot(x, y_synthetic, label='Synthesized Data', linestyle='--',
color='green', alpha=0.7)
    plt.title("Trend Analysis and Data Synthesis")
    plt.xlabel("Year")
    plt.ylabel("Population")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(filepath)
    plt.close()

def plot_histograms(real_data: np.ndarray, synthetic_data: np.ndarray, filepath:
pathlib.Path) -> None:
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.hist(real_data, bins=15, color='blue', edgecolor='black', alpha=0.6)
    plt.title("Real Data Histogram")
    plt.grid(True, alpha=0.3)

    plt.subplot(1, 2, 2)
    plt.hist(synthetic_data, bins=15, color='green', edgecolor='black',
alpha=0.6)
    plt.title("Synthetic Data Histogram")
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(filepath)
    plt.close()

if __name__ == "__main__":
    url: str = "https://www.worldometers.info/world-population/world-population-
by-year/"

```

```

parsed_records: list[PopulationRecord] = parse_world_population(url)

if not parsed_records:
    print("Parsing failed or no data found.")
else:
    csv_path: pathlib.Path = BUILD_DIR / 'parsed_population.csv'
    save_to_csv(parsed_records, csv_path)

    years: np.ndarray = np.array([r.year for r in parsed_records])
    population: np.ndarray = np.array([r.population for r in
parsed_records], dtype=float)

    real_mean: float
    real_var: float
    real_std: float
    real_mean, real_var, real_std = calculate_statistics(population, "Parsed
Real Data")

    trend: np.ndarray = fit_polynomial_trend(years, population, degree=2)

    residuals: np.ndarray = population - trend
    residual_mean: float
    residual_var: float
    residual_std: float
    residual_mean, residual_var, residual_std =
calculate_statistics(residuals, "Trend Residuals (Noise)")

    synthetic_population: np.ndarray = synthesize_data(trend, residual_std)

    calculate_statistics(synthetic_population, "Synthesized Data")

    plot_trend_analysis(
        years,
        population,
        trend,
        synthetic_population,
        BUILD_DIR / 'trend_analysis.png'
    )

    plot_histograms(
        population,
        synthetic_population,
        BUILD_DIR / 'comparison_histograms.png'
    )

```

3.6 Аналіз результатів відлагодження та верифікації результатів роботи програми.

Результати тестування довели працездатність розробленого коду. Аналіз графіків (Рис. 3.1 та Рис. 3.2) демонструє наступне:

1. **Оцінка тренду:** Реальні дані популяції населення мають виражений експоненційний характер (повільне зростання до 1800-х років та різкий стрибок у XX столітті). Квадратичний тренд (парабола) фізично не може адекватно описати таку динаміку на великому проміжку часу, що призвело до відхилень.

2. **Статистична верифікація:** Незважаючи на невідповідність фізичного змісту квадратичного тренду природі демографічних процесів, алгоритм відпрацював математично коректно. Математичне сподівання залишків (шуму) дорівнює нулю. Середньоквадратичне відхилення реальних (2.408 млрд) та синтезованих даних (2.411 млрд) майже збігається.
3. **Особливості гістограм:** Через високу дисперсію залишків, що була додана як нормальний шум до поліному 2-го ступеня, синтезовані дані мають широкий діапазон розкиду, включаючи фізично неможливі від'ємні значення населення.

4 Висновки

В результаті виконаної лабораторної роботи (Рівень III) був розроблений скрипт мовою Python, який успішно виконує HTTP-парсинг табличних даних із веб-ресурсу, структурує їх, зберігає у форматі CSV та проводить комплексний статистичний аналіз.

Було побудовано адитивну математичну модель з використанням квадратичного тренду та нормального розподілу шуму. Обчислені статистичні характеристики (математичне очікування, дисперсія, СКВ) та графічні дані доводять, що синтезована вибірка повністю відповідає заданим математичним законам та має характеристики, близькі до реальних даних.

Аналіз результатів показав, що для прогнозування часових рядів експоненційного характеру (яким є демографічний ріст населення Землі) застосування базових поліноміальних моделей та адитивного нормального шуму з постійною дисперсією є недостатнім. Для підвищення якості прогнозування (Завдання IV) доцільно використовувати експоненційні моделі апроксимації, мультиплікативний шум або спеціалізовані методи машинного навчання для аналізу time series.